A1.1 — Dataset và Protocol thí nghiệm

Bài tập lớn A1: CNN vs Transformer trên tập ảnh lớn

- **Môn:** Học sâu và ứng dụng trong thị giác máy tính (CO5085) — HK261 (2026–2027)
- **Giảng viên:** Lê Thành Sách, Nguyễn Quốc Minh
- **Nhóm:** Hummingbird
- **Thành viên:** Nguyễn Lê Hoàng Hưng (2570214), Nguyễn Thị Linh Trang (2570344)
- **Ngày nộp:** 26/10/2026

---

1. Dataset

1.1 Tập dữ liệu được chọn: Food-101

Thuộc tính	Giá trị
Nguồn	Bossard, Guillaumin, Van Gool, *Food-101 – Mining Discriminative Components with Random Forests*, ECCV 2014
Truy cập	`torchvision.datasets.Food101`
Số lớp	101 món ăn
Kích thước gốc	101.000 ảnh: 750 train + 250 test mỗi lớp
Độ phân giải	Ảnh màu RGB, cạnh dài tối đa 512 px
License	Ảnh lấy từ foodspotting.com, chỉ dùng cho mục đích nghiên cứu/học tập, không thương mại. Nhóm không phân phối lại ảnh; repo chỉ chứa code tải dữ liệu.

1.2 Lý do chọn

1. **Đủ lớn và không quá dễ:** 101 lớp, dạng fine-grained (nhiều món giống nhau về màu sắc và kết cấu, ví dụ *steak* và *filet mignon*). Bài toán khó hơn hẳn CIFAR-10 nên khác biệt giữa CNN và ViT, giữa scratch và pretrained sẽ thấy rõ.
2. **Ảnh tự nhiên, độ phân giải cao:** dùng được trực tiếp ở 224×224, đúng độ phân giải của các mô hình pretrained ImageNet, không cần phóng to ảnh nhỏ như CIFAR-100 (32×32).
3. **Không trùng ImageNet-1k:** khác với Tiny-ImageNet, Food-101 không phải tập con của ImageNet nên kết quả pretrained không bị rò dữ liệu, so sánh scratch và pretrained công bằng hơn.
4. **Có nhiễu nhãn:** split train gốc được tác giả cố ý để lại nhiễu nhãn, còn split test được làm sạch thủ công. Điều này hữu ích cho phần phân tích lỗi.

1.3 Split (subset có kiểm soát)

Do GPU hạn chế, nhóm dùng một subset có kiểm soát của split train và giữ nguyên toàn bộ split test chính thức:

Split	Cách lấy	Số ảnh/lớp	Tổng
Train	Lấy ngẫu nhiên, phân tầng theo lớp, từ split train gốc	300	30.300
Validation	Lấy từ phần còn lại của split train gốc (không giao với Train)	50	5.050
Test	Toàn bộ split test chính thức	250	25.250

- Chia bằng seed cố định `42`. Danh sách ảnh của từng split được lưu vào `A1/splits/{train,val}.txt` và commit lên repo để tái lập chính xác.
- Validation dùng để chọn checkpoint và hyperparameter. Test chỉ dùng **một lần** để báo cáo kết quả cuối.
- Tất cả cấu hình dùng **cùng một split**.
- Nếu còn thời gian GPU, nhóm sẽ chạy thêm với toàn bộ 750 ảnh/lớp và ghi rõ trong báo cáo A1.2.

1.4 EDA dự kiến (trình bày ở A1.2)

- Số ảnh mỗi lớp trên từng split (kiểm tra cân bằng).
- Phân bố kích thước và tỉ lệ khung hình ảnh.
- Lưới ảnh mẫu cho mỗi lớp, kèm một số ví dụ nhiễu nhãn trong split train.

---

2. DataLoader và Augmentation

Chuẩn hóa theo mean/std của ImageNet: `mean = (0.485, 0.456, 0.406)`, `std = (0.229, 0.224, 0.225)`.

Train:

```
RandomResizedCrop(224, scale=(0.35, 1.0))
RandomHorizontalFlip(p=0.5)
TrivialAugmentWide()
ToTensor(), Normalize(mean, std)
RandomErasing(p=0.25)
```

Validation / Test:

```
Resize(256), CenterCrop(224)
ToTensor(), Normalize(mean, std)
```

- Cùng một pipeline augmentation cho mọi cấu hình, để chỉ thay đổi một yếu tố (kiến trúc hoặc chế độ huấn luyện) mỗi lần so sánh.
- Label smoothing `0.1` cho mọi cấu hình.

- DataLoader: `batch_size = 64`, gradient accumulation 2 bước (batch hiệu dụng 128) cho vừa 8GB VRAM; `num_workers = 4`, `pin_memory = True`, shuffle ở train.
- Tiền xử lý một lần: resize toàn bộ ảnh về cạnh ngắn 256 px và lưu lại. Việc này giảm thời gian giải mã JPEG trên CPU, vốn dễ trở thành nút thắt khi ảnh gốc lên tới 512 px.
- Huấn luyện mixed precision (AMP, fp16), chủ yếu để giảm bộ nhớ. GTX 1070 (kiến trúc Pascal) không có Tensor Core nên AMP gần như không tăng tốc; thời gian đo được sẽ được báo cáo đúng như thực tế.

---

3. Mô hình

Chọn hai mô hình có **số tham số tương đương** để so sánh công bằng. Trọng số pretrained lấy từ thư viện `timm`, đều pretrained trên ImageNet-1k.

Họ	Mô hình	Checkpoint `timm`	Tham số (backbone)	Head mới cho 101 lớp
CNN	ResNet-50	`resnet50.a1_in1k`	~23,5M	`Linear(2048 → 101)`
Transformer	DeiT-Small (ViT-S/16)	`deit_small_patch16_224.fb_in1k`	~21,7M	`Linear(384 → 101)`

- Hai mô hình có cùng độ phân giải đầu vào (224) và quy mô tương tự (~22–25M tham số), đều pretrained trên ImageNet-1k. Khác biệt chính là **inductive bias**: tích chập cục bộ so với self-attention toàn cục.
- Head phân loại luôn được khởi tạo lại ngẫu nhiên cho 101 lớp.
- Phương án dự phòng nếu đủ tài nguyên: ConvNeXt-Tiny và Swin-Tiny (~28M tham số).

---

4. Các chế độ huấn luyện

4.1 Ma trận thí nghiệm chính (2 mô hình × 4 chế độ = 8 run)

#	Chế độ	Khởi tạo	Phần được train	ResNet-50	DeiT-S
1	**From-scratch**	Ngẫu nhiên	Toàn bộ	✓	✓
2	**Freeze toàn bộ backbone** (linear probe)	ImageNet	Chỉ head	✓	✓
3	**Freeze một phần**	ImageNet	Nửa sau backbone + head	✓	✓
4	**Full finetune**	ImageNet	Toàn bộ	✓	✓

So sánh from-scratch và pretrained được thực hiện cho **cả hai họ** kiến trúc, trên cùng split.

4.2 Định nghĩa "freeze một phần"

Tiêu chí chung: freeze khoảng **nửa đầu theo độ sâu** của backbone.

Mô hình	Bị freeze	Được train
ResNet-50 (16 bottleneck block: 3-4-6-3)	`conv1`, `bn1`, `layer1`, `layer2` (7/16 block)	`layer3`, `layer4`, `fc`
DeiT-S (12 transformer block)	`patch_embed`, `cls_token`, `pos_embed`, `blocks[0:6]` (6/12 block)	`blocks[6:12]`, `norm`, `head`

- Với ResNet, các lớp BatchNorm trong phần bị freeze được đặt ở chế độ `eval()` để không cập nhật running statistics.
- Ở chế độ 2 (linear probe), toàn bộ backbone ở chế độ `eval()`.
- Số tham số trainable chính xác của từng cấu hình sẽ được đếm bằng code và báo cáo ở A1.2.

4.3 Hyperparameter

Chung cho mọi cấu hình: optimizer AdamW, lịch học cosine có warmup, batch hiệu dụng 128 (64 × 2 bước accumulation), label smoothing 0,1, AMP, seed 42.

Chế độ	Epoch	Warmup	Learning rate (head / backbone)	Weight decay
From-scratch	50	5	1e-3 / 1e-3	0,05
Freeze toàn bộ	15	1	1e-3 / —	1e-4
Freeze một phần	15	1	1e-3 / 1e-4	0,05
Full finetune	15	1	1e-3 / 1e-4	0,05

- From-scratch được nhiều epoch hơn vì cần nhiều bước hơn để hội tụ. Điều này được ghi rõ khi so sánh chi phí.
- Số epoch from-scratch (50) bị giới hạn bởi GPU. Ước tính trên GTX 1070: khoảng 4–5 phút/epoch với 30k ảnh, tức ~4 giờ cho mỗi run from-scratch và ~1 giờ cho mỗi run finetune. Tổng 8 run chính khoảng 12–15 giờ GPU. Nếu dùng thêm GPU Kaggle, nhóm sẽ tăng lên 100 epoch và ghi rõ trong báo cáo.
- Checkpoint được chọn theo accuracy cao nhất trên validation.
- Có thể tinh chỉnh learning rate trên validation; mọi thay đổi so với bảng trên sẽ được ghi lại trong báo cáo A1.2.

4.4 Thí nghiệm phụ (nếu đủ thời gian GPU)

1. **Ảnh hưởng của lượng dữ liệu:** chạy from-scratch và full finetune trên 10%, 50%, 100% tập Train (30, 150, 300 ảnh/lớp) cho cả hai mô hình. Mục đích: kiểm chứng giả thuyết ViT phụ thuộc vào pretrain nhiều hơn CNN khi ít dữ liệu.
2. **Độ ổn định:** chạy lại các cấu hình full finetune với 3 seed (42, 43, 44) và báo cáo mean ± std.

---

5. Đánh giá

5.1 Metric

Nhóm	Metric
Chất lượng	Top-1 accuracy, Top-5 accuracy, Macro-F1 trên validation và test
Chi phí	Số tham số trainable / tổng; thời gian train mỗi epoch và tổng thời gian; bộ nhớ GPU tối đa (`torch.cuda.max_memory_allocated`); throughput lúc inference (ảnh/giây)

5.2 Phân tích

- Learning curve (loss và accuracy trên train/val theo epoch) cho từng cấu hình.
- Confusion matrix trên test của cấu hình tốt nhất mỗi họ, và top-10 cặp lớp hay bị nhầm.
- Lưới ảnh dự đoán sai, phân loại nguyên nhân (món giống nhau, nhiễu nhãn, ảnh nhiều món).
- (Tùy chọn) Grad-CAM cho ResNet-50 và attention map cho DeiT-S trên cùng ảnh.

5.3 Câu hỏi cần trả lời ở A1.2

1. Khi train from-scratch trên ~30k ảnh, CNN hay ViT tốt hơn? Vì sao?
2. Pretrain giúp mỗi họ kiến trúc bao nhiêu? ViT có cần pretrain nhiều hơn CNN không?
3. Khi nào freeze có lợi (tiết kiệm chi phí, ít overfit) và khi nào nên full finetune?
4. Đánh đổi giữa accuracy và chi phí (thời gian, bộ nhớ, tham số trainable) của từng cấu hình.

---

6. Môi trường và tái lập

- **Phần cứng:** máy cá nhân với GPU NVIDIA GeForce GTX 1070 (8GB VRAM, kiến trúc Pascal). Dự phòng: GPU T4 trên Kaggle cho các run from-scratch nếu thiếu thời gian.
- **Phần mềm:** Python 3.12, PyTorch 2.8 (bản CUDA 12.6, vì bản CUDA 12.8 trở lên không còn hỗ trợ GPU Pascal), torchvision 0.23, timm, scikit-learn, matplotlib.
- **Tái lập:** seed cố định; file split được commit; mỗi run có một file config riêng; log từng epoch được lưu lại.
- **Dữ liệu không commit lên repo:** ảnh được tải tự động bằng `torchvision.datasets.Food101` vào thư mục `data/` (đã có trong `.gitignore`).

Cấu trúc thư mục dự kiến

```
A1/
├── configs/      # file yaml cho từng run (model, mode, lr, epochs)
├── docs/         # protocol A1.1, báo cáo A1.2
├── splits/       # train.txt, val.txt
├── src/          # data.py, models.py (hàm freeze), train.py, evaluate.py
├── scripts/      # tạo split, chạy toàn bộ thí nghiệm
├── results/      # log, bảng kết quả, biểu đồ
└── README.md
```

---

7. Kế hoạch thực hiện

Thời gian	Công việc
26/10/2026	**Nộp A1.1:** dataset, split, augmentation, mô hình, ma trận thí nghiệm
27/10 – 01/11	Viết code (data, freeze, train, eval); tạo split; EDA; chạy 6 run pretrained (chế độ 2, 3, 4)
02/11 – 06/11	Chạy 2 run from-scratch; thí nghiệm phụ nếu đủ thời gian
07/11 – 09/11	Tổng hợp bảng, biểu đồ, phân tích lỗi; viết báo cáo PDF; cập nhật GitHub Pages. **Nộp A1.2**